19A04601P MICROPROCESSORS AND MICROCONTROLLERS LAB

Course Objectives:

- Write ALP for arithmetic and logical operations in 8086
- Familiarize with MASM, Embedded C & Code composer studio
- Write and execute programs in 8086, 8051 and ARM Cortex M0

Conduct all the experiments:

List of Experiments:

Intel 8086 (16 bit Micro Processor)

1. Perform simple arithmetic operations using different addressing modes.

2. Sort an array of binary numbers.

3. Code Conversion (Eg. ASCII to Packed BCD form).

4. Addition of an array of BCD numbers stored in packed form.

5. Multiplying two 3x3 matrices and print on DOS

6. Identification & displaying the activated key using DOS & BIOS function calls.

Intel 8051 (8 bit Microcontroller)

1. Detection of key closure (connected to a port line) by polling technique.

2. Delay generation using i) Nested loop & ii) Timers.

3. Counting of external event occurrence through port line

ARM Cortex M0 – NXP LPC Xpress/1115

1. Introduction to the Keil MDK-ARM tool, C and Assembly coding - Processing text in assembly language

2. Configure GPIO for Digital input and output

3. Study of mixed assembly and C programming – Calling a C function from assembly and Calling an assembly function from C

INTRODUCTION TO MASM

The Assembler is a program that converts an assembly input file (source file) to an object file. This object file can further be converted into executable file that is into the machine codes using linker. There are many numbers of assemblers available in the market such as MASM (Macro Assembler), TASM (Turbo Assembler) and NASM (Net wide assembler) etc…

EDITOR:

An editor is a program, which allows you to create a file containing the assembly language statements for your program. As you type in your program, the editor stores the ASCII codes for the letters and numbers in successive RAM locations. When you have typed in all of your programs, you then save the file on a floppy of hard disk. This file is called source file. The next step is to process the source file with an assembler. In the MASM assembler, you should give your source file name the extension, .ASM

ASSEMBLER:

An assembler program is used to translate the assembly language mnemonics for instructions to the corresponding binary codes. When you run the assembler, it reads the source file of your program the disk, where you saved it after editing on the first pass through the source program the assembler determines the displacement of named data items, the offset of labels and pails this information in a symbol table. On the second pass through the source program, the assembler produces the binary code for each instruction and inserts the offset etc that is calculated during the first pass. The assembler generates two files on floppy or hard disk. The first file called the object file is given the extension. OBJ. The object file contains the binary codes for the instructions and information about the addresses of the instructions. The second file generated by the assembler is called assembler list file. The list file contains your assembly language statements, the binary codes for each instruction and the offset for each instruction. In MASM assembler, MASM source file name.ASM is used to assemble the file. Edit source file name LST is used to view the list file, which is generated, when you assemble the file.

LINKER:

A linker is a program used to join several object files into one large object file and convert to an **exe** file. The linker produces a link file, which contains the binary codes for all the combined modules. The linker however doesn't assign absolute addresses to the program, it assigns is said to be relocatable because it can be put anywhere in memory to be run. In MASM, LINK source filename .obj is used to link the file.

DEBUGGER:

A debugger is a program which allows you to load your object code program into system memory, execute the program and troubleshoot are debug it the debugger allows you to look at the contents of registers and memory locations after your program runs. It allows you to change the contents of register and memory locations and return the program. A debugger also allows you to set a break point at any point in the program. If you inset a breakpoint the debugger will run the program up to the instruction where the breakpoint is set and stop execution. You can then examine register and memory contents to see whether the results are correct at that point. In MASM, Debug filename.exe is used to debug the file.

ASSEMBLER DIRECTIVES: The limits are given to the assembler using some pre defined alphabetical strings called Assembler Directives which help assembler to correctly understand. The assembly language programs to prepare the codes.

DB	GROUP	EXTRN
DW	LABEL	TYPE
DQ	LENGTH	EVEN
DT	LOCAL	SEGMENT
ASSUME	NAME	
END	OFFSET	
ENDP	ORG	
ENDS	PROC	
EQU	PTR	

DB-Define Byte: The DB directive is used to reserve byte of memory locations in the available on memory.

DW-Define Word: The DW directive is used to reserve 16 byte of memory location available on memory.

DQ-Define Quad Word (4 words): The DQ directives are used to reserve 8 bytes of memory locations in the memory available.

DT-Define Ten Byte: The DT directive is used to reserve 10 byte of memory locations in the available memory.

ASSUME: Assume local segment name the Assume directive is used to inform the assembler. The name of the logical segments to be assumed for different segment used in programs.

END: End of the program the END directive marks the end of an ALP. ENDP: End of the procedure.

ENDS: End of the segment.

EQU: The directive is used to assign a label with a variable or symbol. The directive is just to reduce recurrence of the numerical values or constants in theprogram.

OFFSET: Specifies offset address.

SEGMENT: The segment directive marks the starting of the logical segment.

END: End of the program the END directive marks the end of an ALP. ENDP: End of the procedure.

ENDS: End of the segment.

EQU: The directive is used to assign a label with a variable or symbol. The directive is just to reduce recurrence of the numerical values or constants in theprogram.

OFFSET: Specifies offset address.

SEGMENT: The segment directive marks the starting of the logical segment.

8086 INSTRUCTION SET SUMMARY:

The following is a brief summary of the 8086 instruction set:

Data Transfer Instructions

MOV	:	Move byte or word to register or memory
IN, OUT	:	Input byte or word from port, output word to port
LEA	:	Load effective address
LDS, LES	:	Load pointer using data segment, extra segment
PUSH, POP	:	Push word onto stack, pop word off stack
XCHG	:	Exchange byte or word
XLAT	:	Translate byte using look-up table

Logical Instructions

NOT	:	Logical NOT of byte or word (one's complement)
AND	:	Logical AND of byte or word
OR	:	Logical OR of byte or word
XOR	:	Logical exclusive-OR of byte or word
TEST	:	Test byte or word (AND without storing)

Shift and Rotate Instructions

SHL, SHR	:	Logical shift left, right byte or word by 1 or CL
SAL, SAR	:	Arithmetic shift left, right byte or word by 1 or CL
ROL, ROR	:	Rotate left, right byte or word by 1 or CL
RCL, RCR	:	Rotate left, right through carry byte or word by 1 or CL

Arithmetic Instructions

ADD, SUB	:	Add, subtract byte or word
ADC, SBB	:	Add, subtract byte or word and carry (borrow)
INC, DEC	:	Increment, decrement byte or word
NEG	:	Negate byte or word (two's complement)
CMP	:	Compare byte or word (subtract without storing)
MUL, DIV	:	Multiply, divide byte or word (unsigned)
IMUL, IDIV	:	Integer multiply or divide byte or word (signed)
CBW, CWD	:	Convert byte to word, word to double word (useful before multiply/divide)
AAA, AAS, AAM, AAD:		**ASCII** adjust for addition, subtraction, multiplication, division (ASCII codes 30-39)
DAA, DAS	:	Decimal adjust for addition, subtraction (binary coded decimal numbers)

Transfer Instructions

JMP	:	Unconditional jump
JA (JNBE)	:	Jump if above (not below or equal)
JAE (JNB)	:	Jump if above or equal (not below)
JB (JNAE)	:	Jump if below (not above or equal)
JBE (JNA)	:	Jump if below or equal (not above)
JE (JZ)	:	Jump if equal (zero)
JG (JNLE)	:	Jump if greater (not less or equal)
JGE (JNL)	:	Jump if greater or equal (not less)
JL (JNGE)	:	Jump if less (not greater nor equal)
JLE (JNG)	:	Jump if less or equal (not greater)

JC, JNC	:	Jump if carry set, carry not set
JO, JNO	:	Jump if overflow, no overflow
JS, JNS	:	Jump if sign, no sign
JNP (JPO)	:	Jump if no parity (parity odd)
JP (JPE)	:	Jump if parity (parity even)
LOOP	:	Loop unconditional, count in CX
LOOPE (LOOPZ)	:	Loop if equal (zero), count in CX
LOOPNE (LOOPNZ)	:	Loop if not equal (not zero), count in CX
JCXZ	:	Jump if CX equals zero

Subroutine and Interrupt Instructions

CALL, RET	:	Call, return from procedure
INT, INTO	:	Software interrupt, interrupt if overflow
IRET	:	Return from interrupt

String Instructions

MOVS	:	Move byte or word string
MOVSB, MOVSW	:	Move byte, word string
CMPS	:	Compare byte or word string
SCAS	:	Scan byte or word string
LODS, STOS	:	Load, store byte or word string
REP	:	Repeat
REPE, REPZ	:	Repeat while equal, zero
REPNE, REPNZ	:	Repeat while not equal (zero)

Processor Control Instructions

STC, CLC, CMC	:	Set, clear, complement carry flag
STD, CLD	:	Set, clear direction flag
STI, CLI	:	Set, clear interrupt enable flag
LAHF, SAHF	:	Load AH from flags, store AH into flags
PUSHF, POPF	:	Push flags onto stack, pop flags off stack

ESC	:	Escape to external processor interface
LOCK	:	Lock bus during next instruction
NOP	:	No operation (do nothing)
WAIT	:	Wait for signal on TEST input
HLT	:	Halt processor

EXECUTION OF ASSEMBLY LANGUAGE PROGRAMMING IN MASM SOFTWARE:

Assembly language programming has 4 steps.

1. EnteringProgram
2. CompileProgram
3. Linking aProgram
4. Debugging aProgram

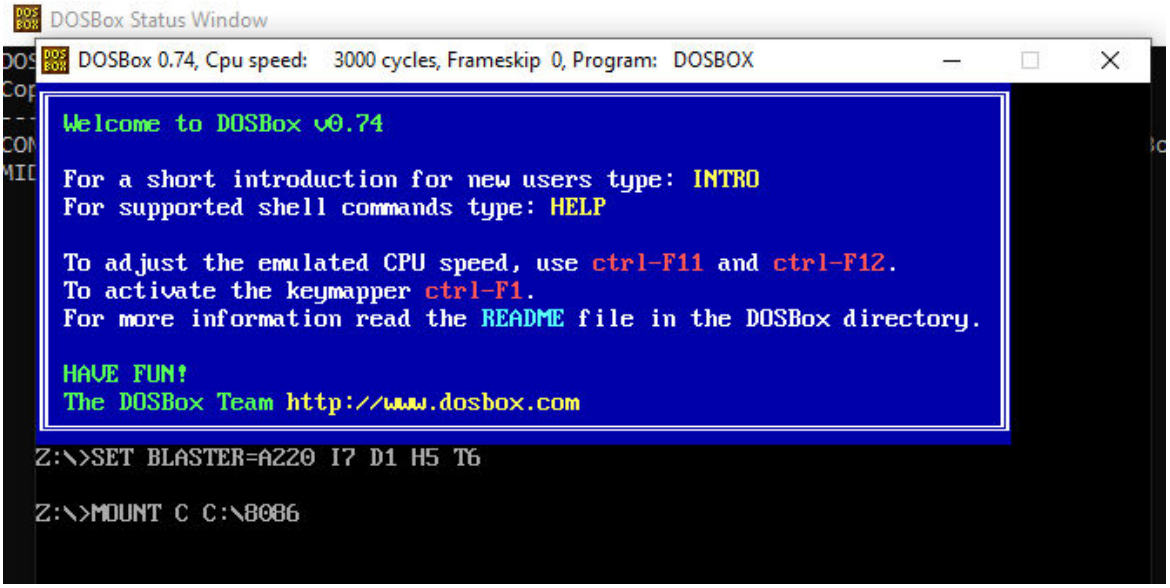
PROCEDURE:

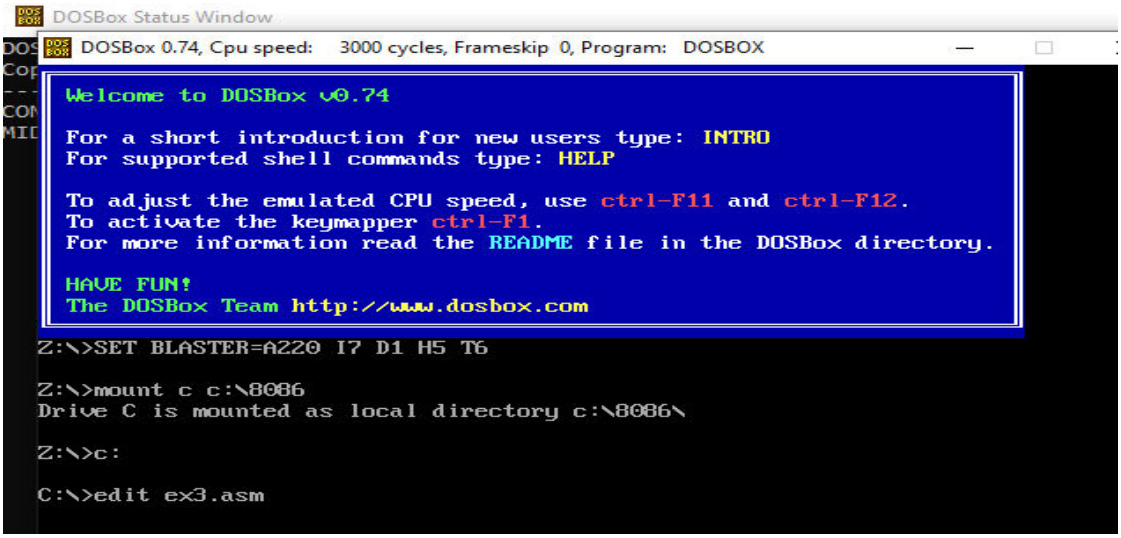
1. Entering Program:-

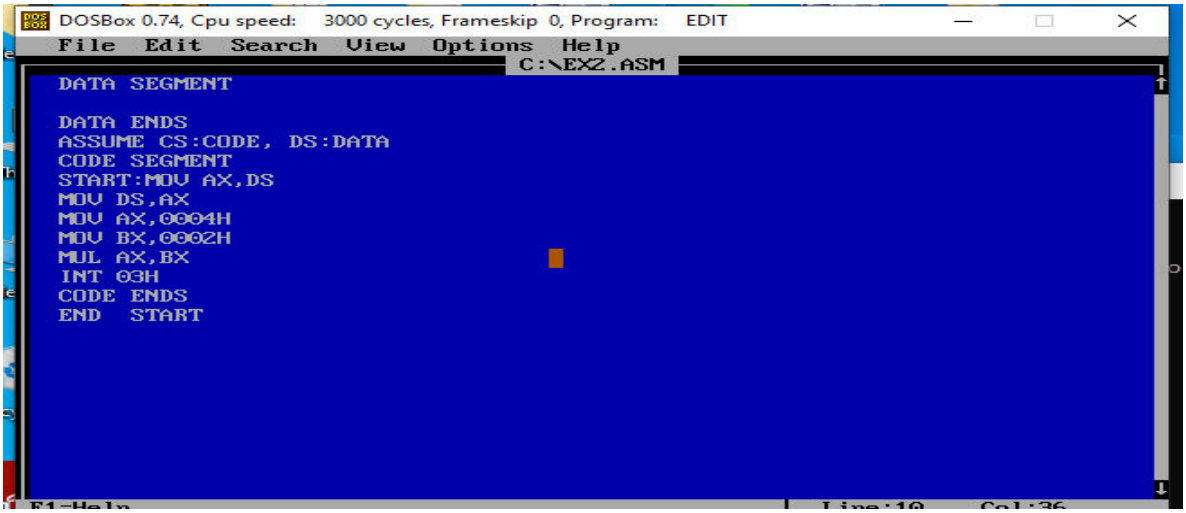
FOR WINDOWS 10

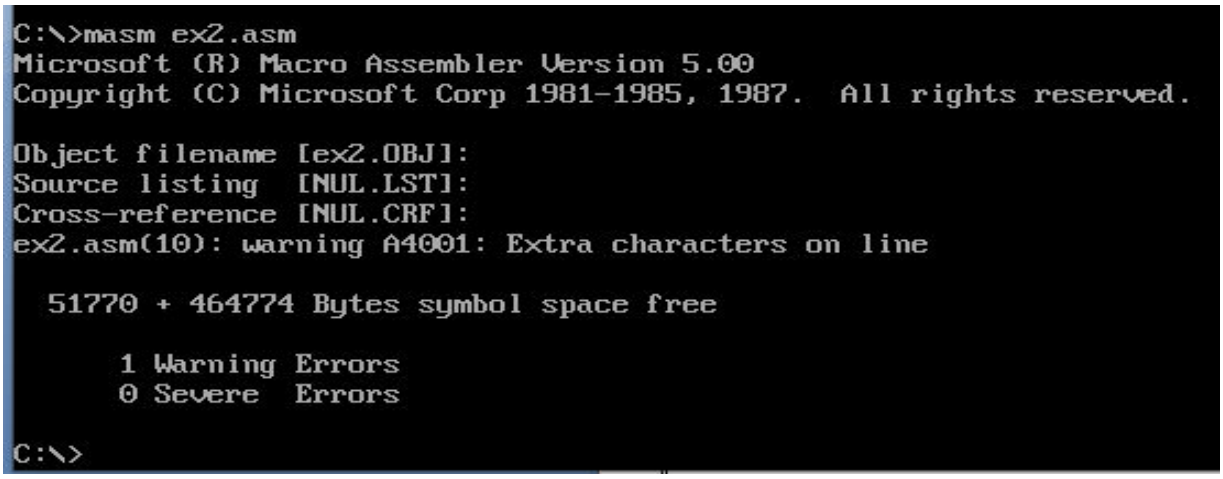
Double click on the DOS BOX icon

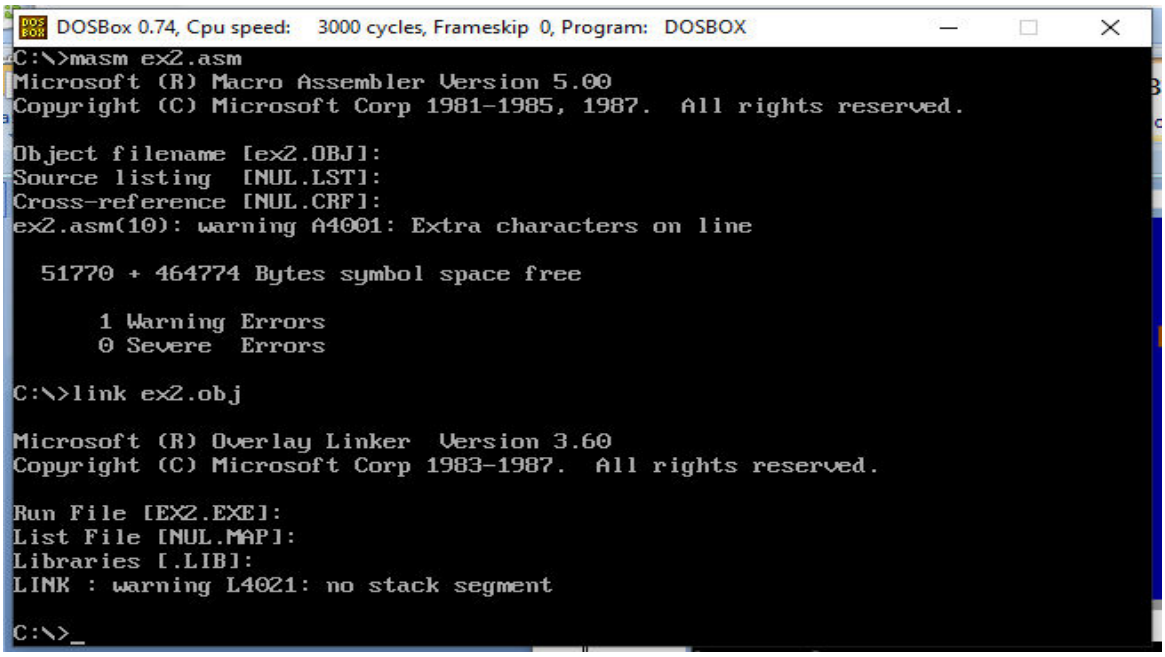
Now type the command as shown below:

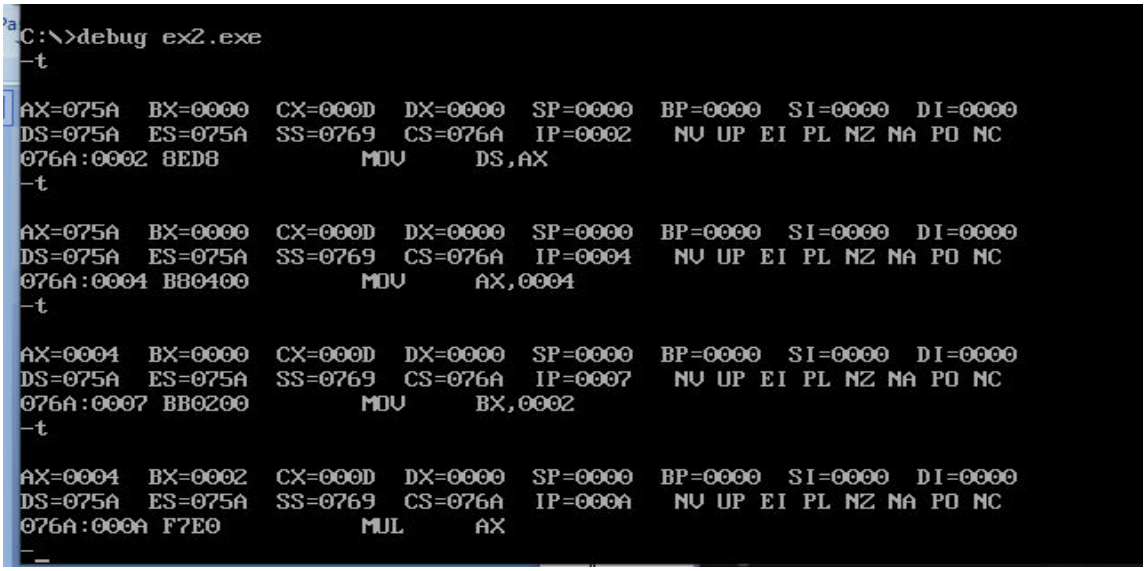



DOSBox Status Window
DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Welcome to DOSBox v0.74
For a short introduction for new users type: INTRO
For supported shell commands type: HELP
To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.
HAVE FUN!
The DOSBox Team http://www.dosbox.com
Z:\>SET BLASTER=A220 I7 D1 H5 T6
Z:\>mount c c:\8086
Drive C is mounted as local directory c:\8086\
Z:\>c:
C:\>edit ex3.asm


DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: EDIT
File Edit Search View Options Help
C:\EX2.ASM
DATA SEGMENT
DATA ENDS
ASSUME CS:CODE, DS:DATA
CODE SEGMENT
START:MOV AX,DS
MOV DS,AX
MOV AX,0004H
MOV BX,0002H
MUL AX,BX
INT 03H
CODE ENDS
END START


C:\>masm ex2.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.
Object filename [ex2.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:
ex2.asm(10): warning A4001: Extra characters on line
51770 + 464774 Bytes symbol space free
1 Warning Errors
0 Severe Errors
C:\>



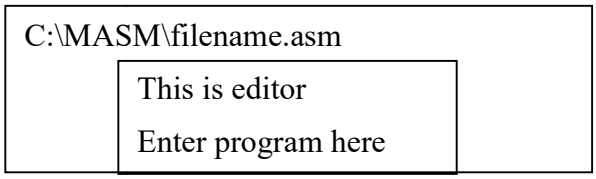


Observe the values
of AX,BX Registers.

FOR WINDOWS7:

StartMenu
Run ↵
Cmd ↵
C:\cd MASM ↵

C:\ MASM> edit filename.asm↵



After entering program save & exit (ALT F & Press S or ALT F &Press X) C:\MASM>

2. Compile the Program:-

C:\MASM>MASMfilename.asm

Microsoft @macro assembler version 5.10 Copy rights reserved© Microsoft

Corp 1981 All rights reserved Object filename [OBJ];↵

List filename [NUL, LIST];↵

Cross Reference [NUL, CRF];↵

Press enter the screen shows c>

3.Linking a Program:-

c> link filename.obj ↵

Microsoft @ overlay linker version 3.64 Copy rights reserved© Microsoft corp.

1983-88. All rights reserved Object module [.OBJ];

Run file [.EXE];↵

List [NUL MAP];↵

Libraries [LIB];↵

Press enter till screen chows c>

4. Debug a Program:-

C> debug filename.exe ↵

- (Screen shows onlydash)

→ **U** To view the contents of the program understood by the Assembler(unassemble).

→**T** To trace the program step by step.

→G go (complete execution).

→**q** To quit from the assembler.

The frequently used command are explained given below.

1. **EDIT(E)** This command is used to examine and modify the content of memory locations. The format of the command is given below.

   ESSSS:DDDD

   SSSS is data segment address and is optional.

   DDDD is offset address.

   If SSSS is not specified it will take the segment address currently available in DS register.

The command displays content of a memory location specified; it can be modified as required.

Press space bar for examining the subsequent locations.

2. **DISPLAY(D)** This command displays the content of a range of memory locations.

   The format of command is given below.

   D SSSS: DDDD, XXXX

   SSSS is data segment address and is optional

   DDDD is starting address of memory range

   XXXX is ending address of memory range

   If only starting address is specified, 128 locations will be displayed.

3. **TRACE (T)** This command is used for executing the program in parts. You can either execute one instruction at a time or a group of instructions at a time.

   The format of command is given below

   T = SSSS: DDDD N

   SSSS is starting address of the code segment of program available in memory. It is optional. If not given, it will take the address from CS register.

   DDDD is the offset of the instruction of the program, it is optional. If not given program will be executed from address available in IP.

   N is the number of instruction to be executed. It is optional. If not given, will be taken as 1.

   After executing the program instructions, content of all registers are displayed on the screen along with the instruction to be executed next.

   Press just T if address is not to be specified and to be taken as default value.

4. **GO (G)** This program is used for executing program in full. It will execute program till INT3 instruction.

   The format of command is given below

   G = SSSS: DDDD

   SSSS is starting address of the code segment of the program available in memory. It is optional.

   DDDD is the offset address of the program. It is optional. If not given, it will be assumed as zero.

   After executing the program content of all registers are displayed on the screen along with the instruction to be executed next.

   Press just G if address is not to be specified and to be taken as default value.

5. **QUIT (Q)** To quit the Debug program press Q followed by CR key.

Advantages of Assembly Language :

An understanding of assembly language provides knowledge of:

- Interface of programs with OS, processor and BIOS;
- Representation of data in memory and other external devices;
- How processor accesses and executes instruction;
- How instructions accesses and process data;
- How a program access external devices.
- It requires less memory and execution time;
- It allows hardware-specific complex jobs in an easier way;

- It is suitable for time-critical jobs;
- It is most suitable for writing interrupt service routines and other memory resident programs.

EXP 1:PERFORM SIMPLE ARITHMETIC OPERATIONS USING DIFFERENT ADDRESSING MODES.

AIM: To write an assembly language program to Perform simple arithmetic operations using different addressing modes.

APPARTUS: MASM Software and PC.

PROGRAM:

a) REGISTER MODE

```
DATA SEGMENT

DATA ENDS

Assume CS: code, DS: DATA
Code segment
Start: MOV AX, 4343
MOV BX, 1111H
ADD AX, BX
INT 03H
Code ends
End start
```

a) INDIRECT MODE

```
DATA SEGMENT

DATA ENDS

Assume CS: code, DS: DATA
CODE SEGMENT
Start: MOV SI, 2000
MOV AX, [SI]
ADD SI, 02
MOV BX, [SI]
ADD AX, BX
MOV DI, 3000
MOV [DI], AX
INT 03
Code ends
End start
```

a) REGISTER MODE

```
DATA SEGMENT

DATA ENDS

Assume CS: code, DS: DATA
```

```
Code segment
Start: MOV AX, 4343
MOV BX, 1111
SUB AX, BX
INT 3
Code ends
End start
```

b) INDIRECT MODE

```
DATA SEGMENT

DATA ENDS

Assume CS: code, DS: DATA
Code segment
Start: MOV SI, 2000
MOV AX, [SI]
ADD SI, 02
MOV BX, [SI]
SUB AX, BX
MOV DI, 3000
MOV [DI], AX
INT 03
Code ends
End start
```

a) REGISTER MODE

```
DATA SEGMENT

DATA ENDS

Assume CS: code, DS: DATA
Code segment
Start: MOV AX, 0040
MOV BX, 0002
MUL BX
INT 3
Code ends
End start
```

b) INDIRECT MODE

```
DATA SEGMENT

DATA ENDS

Assume CS: code, DS: DATA
```

```
Code segment
Start: MOV SI, 2000
MOV AX, [SI]
ADD SI, 02
MOV BX, [SI]
MUL BX
MOV DI, 3000
MOV [DI], AX
INT 03
Code ends
End start
```

a) REGISTER MODE

```
DATA SEGMENT

DATA ENDS

Assume CS: code, DS: DATA
Code segment
Start: MOV AX, 0040
MOV BX, 0002
DIV BX
INT 3
Code ends
End start
```

b) INDIRECT MODE

```
DATA SEGMENT

DATA ENDS

Assume CS: code, DS: DATA
Code segment
Start: MOV SI, 2000
MOV AX, [SI]
ADD SI, 02
MOV BX, [SI]
DIV BX
MOV DI, 3000
MOV [DI], AX
INT 03
Code ends
End start
```

RESULT:

EXP2: Sort an ARRAY of binary numbers

AIM: To write an assembly language program to Perform sort an array of binary numbers

APPARTUS: MASM Software and PC.

```
DATA SEGMENT
LIST DW 05H, 04H, 01H, 03H, 02H
COUNT EQU 05H
DATA ENDS
CODE SEGMENT
ASSUME CS : CODE, DS : DATA

START: MOV AX, DATA
MOV DS, AX
MOV DX, COUNT - 1
BACK : MOV CX, DX
MOV SI,OFFSET LIST
AGAIN : MOV AX, [SI]
CMP AX, [SI + 2]
JC GO
XCHG AX, [SI + 2]
XCHG AX, [SI]
GO: INC SI
INC SI
LOOP AGAIN
DEC DX
JNZ BACK
INT 03
CODE ENDS
END START
```

RESULT:

Input:

05H, 04H, 01H, 03H, 02H

Output:

01H, 02H, 03H, 04H, 05H

```
DATA SEGMENT
LIST DW 03H, 04H, 01H, 05H, 02H
COUNT EQU 05H
DATA ENDS
CODE SEGMENT
ASSUME CS : CODE, DS : DATA
START: MOV AX, DATA
MOV DS, AX
MOV DX, COUNT - 1
BACK : MOV CX, DX
MOV SI, OFFSET LIST
AGAIN : MOV AX, [SI]
CMP AX, [SI + 2]
JNC GO
XCHG AX, [SI + 2]
XCHG AX, [SI]
GO:INC SI
INC SI
LOOP AGAIN
DEC DX
JNZ BACK
HLT
CODE ENDS
END START
```

RESULT:

Input:
03H, 04H, 01H, 05H, 02H
Output:
05H, 04H, 03H, 02H, 01H

OTHER METHOD FOR ASCENDING & DESCENDING

ASCENDING

```
DATA SEGMENT
STRING1 DB 99H,12H,56H,45H,36H
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE,DS:DATA
START: MOV AX,DATA
MOV DS,AX

MOV CH,04H

UP2: MOV CL,04H
LEA SI,STRING1

UP1: MOV AL,[SI]
MOV BL,[SI+1]
CMP AL,BL
```

```
JC DOWN
MOV DL,[SI+1]
XCHG [SI],DL
MOV [SI+1],DL

DOWN: INC SI
DEC CL
JNZ UP1
DEC CH
JNZ UP2

INT 3
CODE ENDS
END START
```

DESCENDING

```
DATA SEGMENT
STRING1 DB 99H,12H,56H,45H,36H
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE,DS:DATA
START: MOV AX,DATA
MOV DS,AX
MOV CH,04H
UP2: MOV CL,04H
LEA SI,STRING1

UP1:MOV AL,[SI]
MOV BL,[SI+1]
CMP AL,BL
JNC DOWN
MOV DL,[SI+1]
XCHG [SI],DL
MOV [SI+1],DL

DOWN: INC SI
DEC CL
JNZ UP1
DEC CH
JNZ UP2
INT 3
CODE ENDS
END START
```

RESULT:

EXP3: CODE CONVERSIONS

AIM: To write an assembly language program to Perform CODE CONVERSIONS

APPARTUS: MASM Software and PC.

PROGRAM:

Code Conversion (Eg. ASCII to Packed BCD form).
BCD to Binary conversion

```
DATA SEGMENT
NO1 DB "9036"
D2 DB ?
D1 DB 16 DUP(?)
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
MOV AX, DATA
MOV DS, AX
LEA SI, NO1
LEA DI, D1
MOV CX, 04H
TOP:
MOV AL, [SI]
MOV DX, 0CH
UP1:
ROL AX,1
DEC DX
JNZ UP1
AND AX, 1111000000000000B
MOV DX, 04H
UP2:
ROL AX, 1
JNC DN
MOV BX, 1
MOV [DI], BX
JMP DN2
DN:
MOV BX, 0
MOV [DI], BX
DN2:
INC DI
DEC DX
JNZ UP2
INC SI
DEC CX
JNZ TOP
INT 3
CODE ENDS
END START
```

BINARY TO BCD CONVERSION

```
DATA SEGMENT
NO1 DB "1001000000110110"
D1 DW 4 DUP (?)
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
MOV AX, DATA
MOV DS, AX
LEA SI, NO1
LEA DI, D1
MOV CX, 04H
TOP:
MOV BX, 00H
MOV AX, [SI]
ROR AX, 1
JNC P2
ADD BX, 08H
P2:
INC SI
MOV AX, [SI]
ROR AX, 1
JNC P3
ADD BX, 04H
P3:
INC SI
MOV AX, [SI]
ROR AX, 1
JNC P4
ADD BX, 02H
P4:
INC SI
MOV AX, [SI]
ROR AX, 1
JNC P5
ADD BX, 01H
P5:
MOV [DI], BX
INC DI
INC SI
DEC CX
JNZ TOP
INT 3
CODE ENDS
END START
```

CONVERT BCD TO SEVEN SEGMENT CODE CONVERSION

```
DATA SEGMENT
TABLE DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH
X       DB      05
ANS     DB      ?
DATA ENDS
        ASSUME CS:CODE, DS: DATA
CODE SEGMENT
START:MOV AX, DATA
MOV DS, AX
        MOV BX, OFFSET TABLE
        MOV AL,X
        XLAT
        MOV ANS, AL
        INT 03H
CODE ENDS
END START
```

Binary to Gray

```
DATA SEGMENT
X       DB      05
ANS     DB      ?
DATA ENDS
        ASSUME CS:CODE, DS: DATA
CODE SEGMENT
START:MOV AL,X
MOV AH,AL
SAL AH,1
XOR AL,AH
MOV AH,AL
MOV AL,X
AND AL,80
OR AL,AH
MOV ANS, AL
CODE ENDS
END START
```

TWO DIGIT PACKED BCD NUMBER INTO TWO UN-PACKED BCD NUMBERS AND THEN TO FIND ASCII EQUIVALENTS OF THE TWO DIGITS.

```
DATA SEGMENT
                PBCD            DB              63H
                DIG1                DB                  ?
                DIG2                DB                  ?
                ASC1                DB                  ?
                ASC2                DB                  ?
DATA    ENDS
CODE            SEGMENT
ASSUME CS: CODE, DS: DATA
START:                  MOV AX, DATA
                        MOV DS, AX
                        MOV AL, PBCD
```

```
        AND AL, 0FH
        MOV DIG1, AL
        ADD AL, 30H
        MOV ASC1, AL
        MOV AL, PBCD
        AND AL, 0F0H
        MOV CL, 04H
        ROL AL, CL
        MOV DIG2, AL
        ADD AL, 30H
        MOV ASC2, AL
INT 03H
CODE ENDS
END START
```

RESULT:

EXP4: ADDITION OF BCD NUMBERS

AIM: To write an assembly language program to Perform BCD NUMBERS

APPARTUS: MASM Software and PC.

PROGRAM:

```
MOV   CX, 0000
      MOV   AX, 1111
      MOV   BX, 2222
      ADD   AX, BX
      JNC   NEXT
      INC   CX
NEXT: MOV   DX, AX
      MOV   EX, CX
      INT 03H
```

RESULT:

EXP5: MULTIPLYING TWO 3X3 MATRICES AND PRINT ON DOS

AIM: To write an assembly language program to Perform Multiplying two 3x3 matrices and print on DOS

APPARTUS: MASM Software and PC.

PROGRAM:

```
ASSUME CS: CODE
CODE SEGMENT
MOV SI,1000
MOV BP,1020
MOV DI,1050
L2: MOV CX,00
L1: MOV AL,[SI]
MOV BL,[BP]
MUL BL
ADD CX,AX
ADD BP,03
INC SI
CMP BP,1029
JB L1
SUB SI,03
SUB BP,08
ADD DI,02
CMP BP,1023
JB L2
ADD SI,03
SUB BP,03
CMP DI,1051
JB L2
HLT
CODE ENDS
END START
```

RESULT:

EXP6: IDENTIFICATION & DISPLAYING THE ACTIVATED KEY USING DOS & BIOS FUNCTION CALLS

AIM: To write an assembly language program to Perform Identification & displaying the activated key using DOS & BIOS function calls
APPARTUS: MASM Software and PC.
PROGRAM:

```
.MODEL SMALL
.CODE
BACK:
LAST:
MOV AX, @DATA ; INITIALIZE THE ADDRESS OF DATA
MOV DS, AX ; SEGMENT IN DS
MOV AH, 01H ; LOAD FUNCTION NUMBER
INT 21H ; CALL DOS INTERRUPT
CMP AL,'0'
JZ LAST ; DISPLAY THE KEYS UNTIL 0 KEY IS PRESSED
JMP BACK
MOV AH, 4CH ; TERMINATE THE
PROGRAM INT 21H
END ; END PROGRAM
```

```
MODEL SMALL
.DATA
.CODE
MSG DB "KEYBOARD WITH BUFFER:",'$' ; MESSAGE FOR THE INPUT
BUFF DB 25
DB 00
DB 25 DUP (?)
MOV AX, @DATA ; INITIALIZE THE ADDRESS OF DATA
MOV DS, AX ; SEGMENT IN DS
MOV AH, 09H
MOV DX, OFFSET MSG ; FUNCTION TO DISPLAY
INT 21H
MOV AH, 0AH
MOV DX, OFFSET BUFF ; FUNCTION TO TAKE BUFFERED DATA
INT21H
MOV AH, 4CH ; TERMINATE THE
PROGRAM INT 21H
END ; END PROGRAM
```

RESULT:

8051

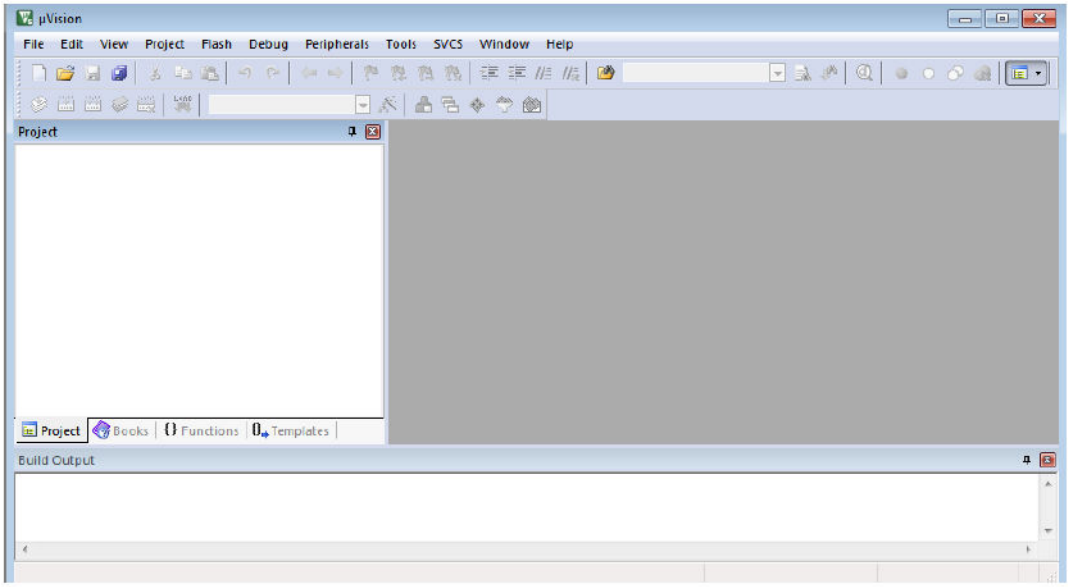
STEPS TO USE KEIL:

1. **Keil Micro vision 3**

Procedure to start up with Keil Micro Vision 3

Starting Micro vision 3

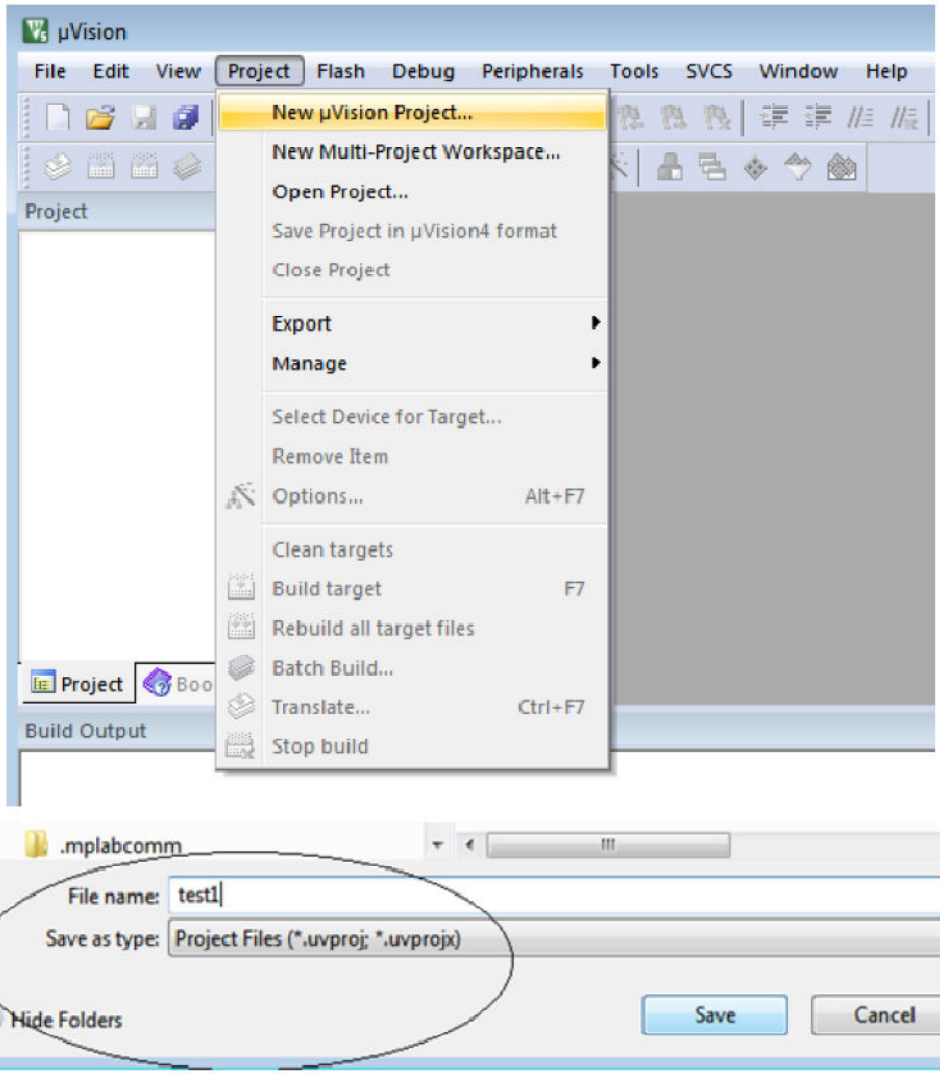
Click on keil Micro Vision icon on the desktop



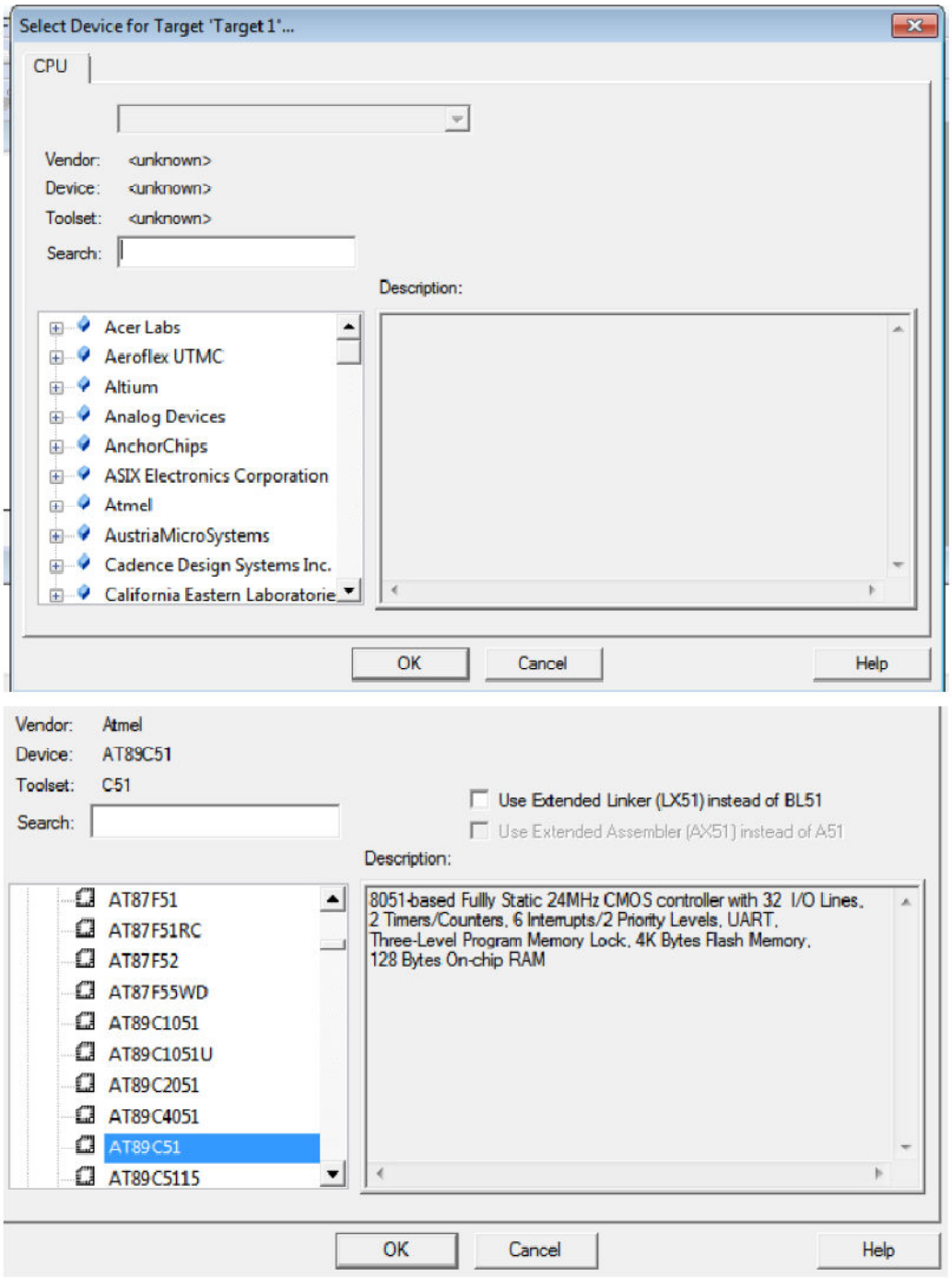
b) Loading a project into Micro Vision 3

Click on Project menu, Select Close Project if any Projects are Present or Select New Project from

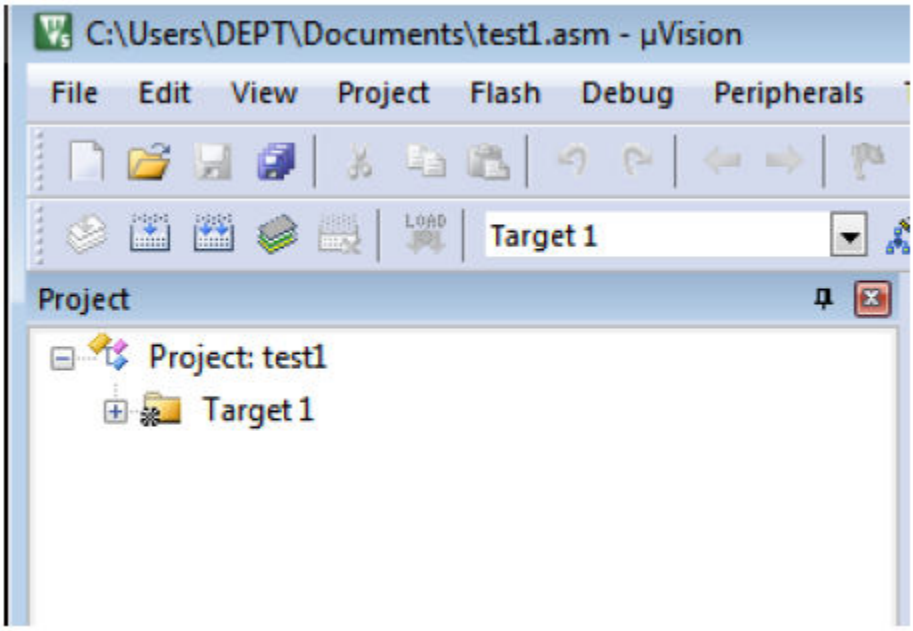
the drop – down menu. Enter the filename and Click on OK



Double Click on ATMEL from the wizard then select AT89c51 and Press OK

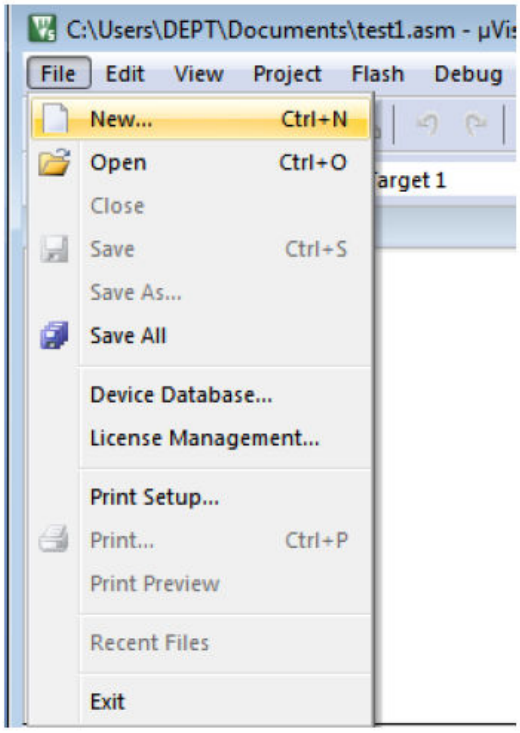


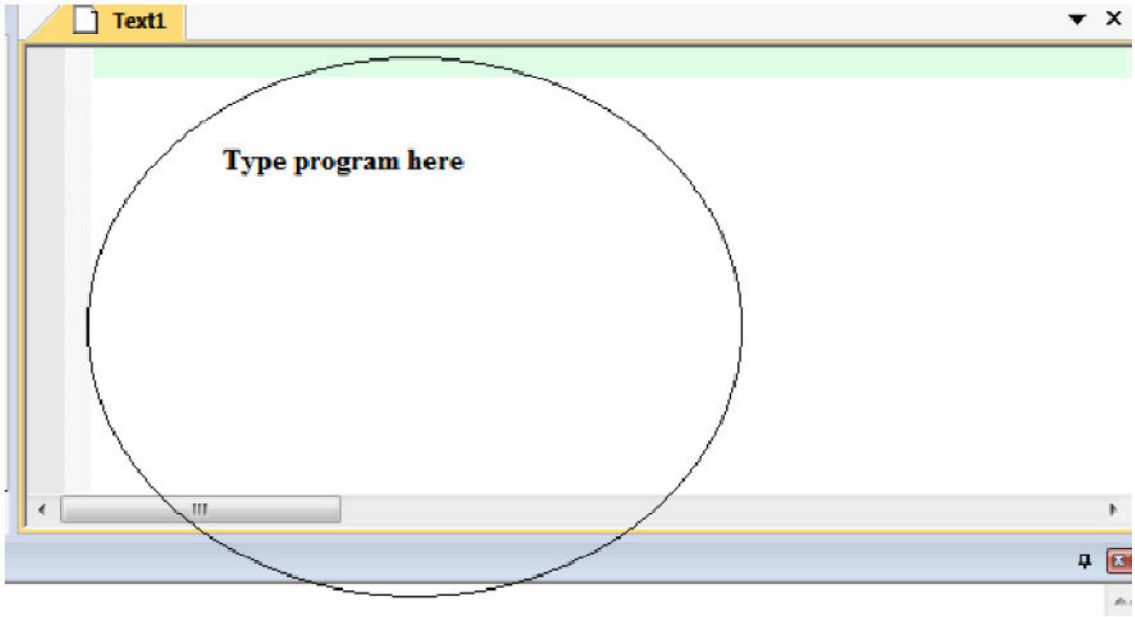
Micro vision 3 will load 8051 Microcontroller Projects file and Display as :

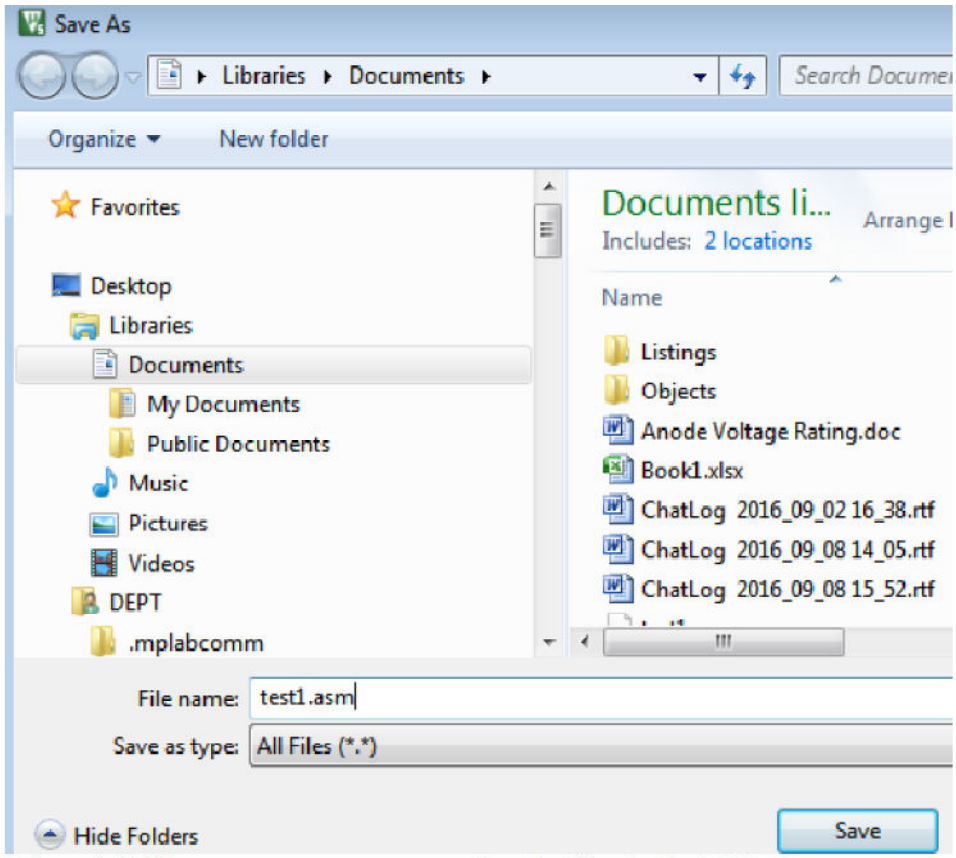


c) Editing and Assembling

Type the program in the work space window. Now save the file and right click source group 1

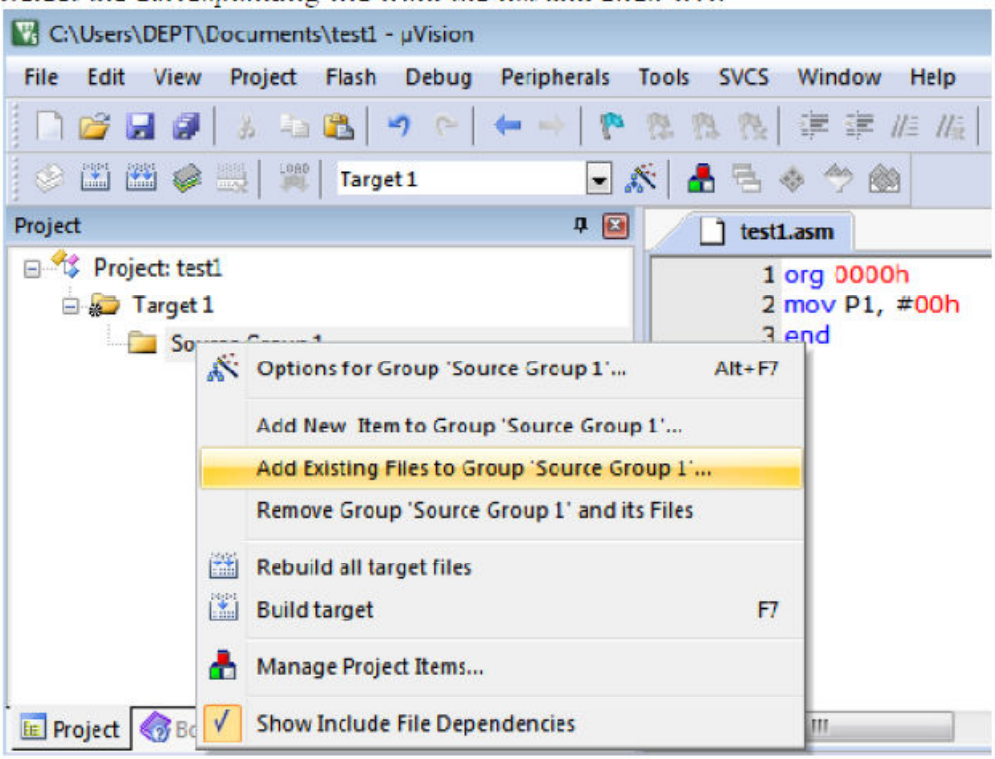






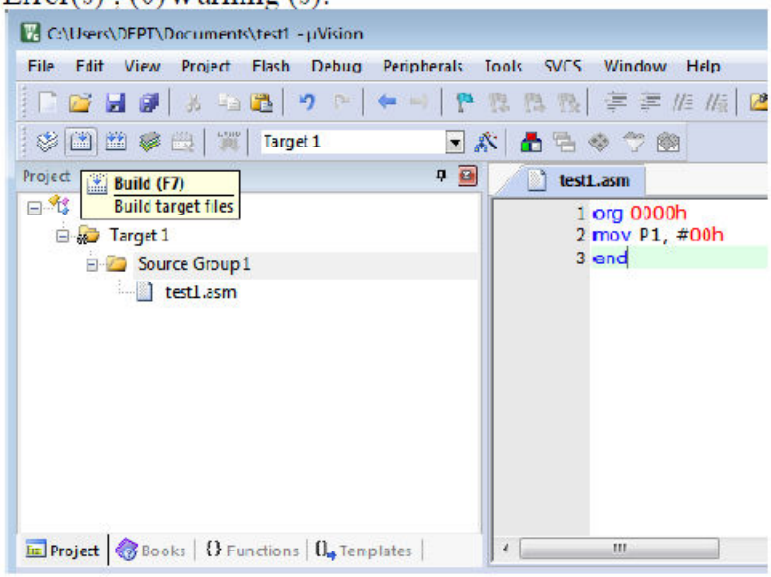
Select Add files to group source group Let the files be in ASM.

Select the corresponding file from the list and click OK.



To assemble select build target, if no error(s) are found the output window will display.

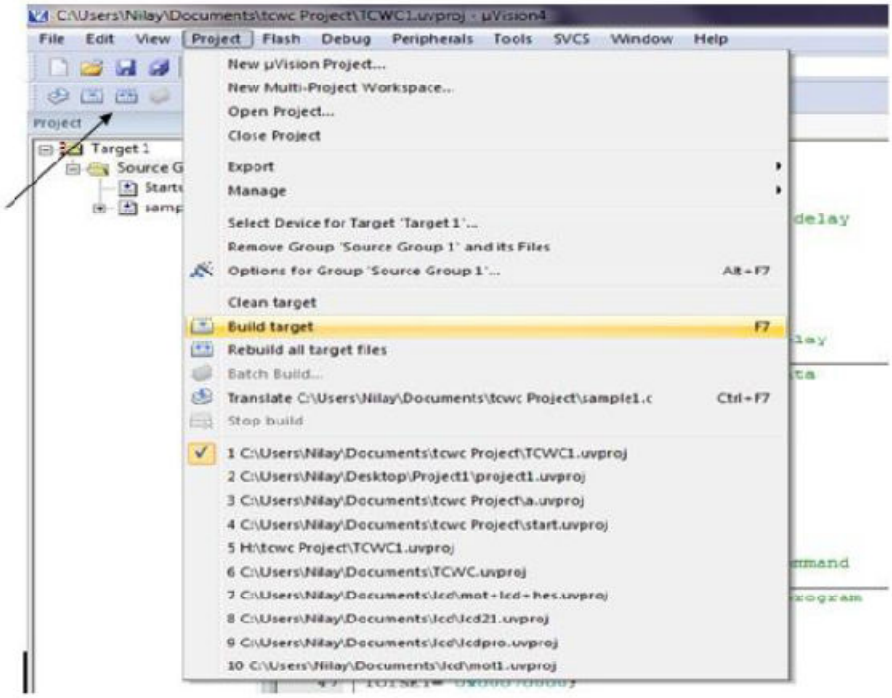
(0) Error(s) . (0)Warning (s).



If error(s) are found then select Rebuild Target and then the Programmer will find it easy to correct the error(s).

d) Debugging

To debug Click on debug button.



EXP7: Detection of key closure (connected to a port line) by polling technique.
Aim: Write program for Detection of key closure (connected to a port line) by polling technique**.**
Software: PC,KEIL
PROGRAM:

```
#include<reg51.h>
void main(void)
   {
      loop:P1=0xFF;              // send all 1's to P1
      while(P1==0xFF);           // remain within loop till key is not pressed
       switch(P1)                // when key pressed detect is
          {
             case 0xFE:
                  P0=0xF9; // and display digit from 1 to 8
                  break;
              case 0xFD:
                  P0=0xA4;
                  break;
              case 0xFB:
                  P0=0xB0;
                  break;
              case 0xF7:
                  P0=0x99;
                   break;
              case 0xEF:
                  P0=0x92;
                  break;
               case 0xDF:
                  P0=0x82;
                   break;
              case 0xBF:
                   P0=0xF8;
                   break;
               case 0x7F:
                   P0=0x80;
                   break;
                }
            goto loop;
      }
```

RESULT:

EXP8: Delay generation using i) Nested loop & ii) Timers.

Aim: Write program for Delay generation using i) Nested loop & ii) Timers.

Software: PC,KEIL

PROGRAM:

```
#include<reg51.h>
#define ON 1
#define OFF 0

sbit led=P2^0;

unsigned int i=0;

void delay();

void delay()
	{
		TMOD = 0x01;    // Timer0 mode1
		TH0 = 0xFC;   //initial value for 1ms
		TL0 = 0x66;
		TR0 = 1;      // timer start
		while(TF0==0);  // check overflow condition
		TR0 = 0;     // Stop Timer
		TF0 = 0;    // Clear flag
	}

void main()
	{
		while(1)
			{
				led = ON;
				for(i=0;i<1000;i++)
				{
					delay();
				}
				led = OFF;
				for(i=0;i<1000;i++)
				{
					delay();
				}
			}
	}
```

RESULT:

EXP9: Counting of external event occurrence through port line

Aim: Counting of external event occurrence through port line

Software: PC,KEIL

PROGRAM:

```
#include <reg51.h>
void main(void)
{
unsigned int x;
for (;;) //repeat forever
{
p1=0x55;
for (x=0;x<40000;x++); //delay size
//unknown
p1=0xAA;
for (x=0;x<40000;x++);
}
}
```

RESULT:

ARM PROGRAMMING

EXP10 : ASSEMBLY LANGUAGE PROGRAM TO PERFORM 32 BIT ADDITION

AIM: To write an assembly language program to Perform 32 Bit Addition

APPARTUS: KEIL Software and PC.

```
MOV R0, #0x40000000
LDR R1, [R0]
ADD R0, R0, #04
LDR R2, [R0]
ADDS R1, R1, R2
ADD R0, R0, #04
 STR R1, [R0]
 SAME B
SAME
END
```

32 Bit Addition using C Language:

```
#include <lpc214x.h>
 void main (void)
{
unsigned char x, y, z ;
 x=0x34;
y=0xA9;
IO0PIN=0xffffffff;
 z=x+y;
IO0PIN=z;
}
```

RESULT:

EXP11: CONFIGURE GPIO FOR DIGITAL INPUT AND OUTPUT

Aim : Write a program to Configure GPIO for Digital Input and Output.

Software: PC,KEIL

PROGRAM:

```
#include<lpc214x.h> void delay();
void main()
{
roodir |=0xffffffff;// port 0 is now acting as a output pin while(1)
{
ioseto |=0xffffffff;// port 0's all pins are high now (led is glowing) delay();
ioclro |=0xffffffff;// port 0's all pins are low now (led is off) delay();
}
void delay()
{
unsigned int i; for(i=0;i<30000;i++);
}
```

RESULT:

EXP 12: Study of mixed assembly and C programming – Calling a C function from assembly and Calling an assembly function from C

Aim : Write a program to Study of mixed assembly and C programming – Calling a C function from assembly and Calling an assembly function from C.

Software: PC,KEIL

PROGRAM:

To call an assembly function from C:

1. In the assembly source, declare the code as a global function using .global and .type the following:

```
.global  myadd
    .p2align 2
    .type    myadd,%function
myadd:                  // Function "myadd" entry point.
    .fnstart
    add      r0, r0, r1   // Function arguments are in R0 and R1. Add together
                          // and put the result in R0.
    bx       lr           // Return by branching to the address in the link
                          // register.
    .fnend
```

2. In C code, declare the external function using extern:

```
#include <stdio.h>
extern int myadd(int a, int b);
int main()
{
  int a = 4;
  int b = 5;
  printf("Adding %d and %d results in %d\n", a, b, myadd(a, b));
```

```
 return (0);
}
```

3. Ensure that your assembly code complies with the Procedure Call Standard for the Arm Architecture (AAPCS). The AAPCS describes a contract between caller functions and callee functions. For example, for integer or pointer types, it specifies that:

   Registers R0-R3 pass argument values to the callee function, with subsequent arguments passed on the stack.

   Register R0 passes the result value back to the caller function.

   Caller functions must preserve R0-R3 and R12, because these registers are allowed to be corrupted by the callee function.

   Callee functions must preserve R4-R11 and LR, because these registers are not allowed to be corrupted by the callee function.

4. Compile both source files:

   armclang --target=arm-arm-none-eabi -march=armv8-a main.c myadd.s

RESULT: